

《组成原理》指令集实验指导手册

向量点积：ARM vs RISC-V 指令集对比实验



姓名：吕昊远

学号：2410668

班号：1012

1. 实验目的

本实验旨在通过实现基础的向量点积算法，观察分析 ARM 与 RISC-V 两种架构在访存寻址模式上的差异。通过对比核心循环体的汇编代码与指令，验证两种指令集在处理加载数据与更新指针这些常见操作时的指令逻辑，从程序运行的底层角度理解指令集的原理与差异，从而更加全面地认知指令效率与代码密度之间的权衡关系。

2. 问题描述：实验背景

在人工智能领域，向量点积运算是大模型训练与推理的核心步骤。该操作要求计算机高频地从内存中读取连续的浮点或定点数据数组，进行相乘并累加。随着人工智能的飞速发展，模型越来越庞大，计算需求与日俱增，对于底层计算的优化，是“加速大概率事件”这一理念的集中体现。

在指令集的学习中，向量点积算法尤其适合进行指令集的对比实验。向量点积的核心是一个紧凑的循环体，其中包含大量的读取数组元素与移动数组指针这两个操作，在不同的指令集下，更方便集中对比原理差异：ARM 指令集拥有相对丰富的寻址模式，支持在一条指令中同时完成访存与指针更新；而 RISC-V 则是秉持极简的设计，将寻址模式与访存分离开来。这种设计理念的差异会在点积计算的循环体代码长度和指令条数上直观地体现出来，在差异较为集中的同时，还更易于直观的对比与分析。

3. 实验环境准备

- 实验工具：QEMU 用户态模拟器（QEMU User 6.2.0）

本次实验推荐在 Linux (Ubuntu 22.04+) 环境下进行，实验过程中需要使用 QEMU User 6.2.0 进行交叉编译：

ARM 编译与运行：arm-linux-gnueabihf-gcc, qemu-arm

RISC-V 编译与运行：riscv64-linux-gnu-gcc, qemu-riscv64

- 环境配置：

使用 apt 安装 QEMU，也可使用如下命令拉取资源并安装：

```
sudo apt update
sudo apt install -y build-essential pkg-config python3 ninja-build \
libgl2.0-dev libpixmap-1-dev wget

cd /tmp
wget https://download.qemu.org/qemu-6.2.0.tar.xz
tar -xf qemu-6.2.0.tar.xz
cd qemu-6.2.0
mkdir build
cd build

../configure --prefix=/opt/qemu-6.2 --target-list=riscv32-linux-
user,riscv64-linux-user,arm-linux-user,aarch64-linux-user
make -j"$(nproc)"

sudo make install
/opt/qemu-6.2/bin/qemu-riscv64 --version
```

- 代码资源：

实验中需要编写算法对应的汇编代码，可直接使用文件中的 1012-2410668-吕昊远-arm.s 与 1012-2410668-吕昊远-riscv.s 进行实验，也可参照代码自行编写，核心部分如下：

ARM：

```

01 dot_product_loop:
02     CMP r3, #0          // 检查计数器
03     BLE end_loop       // 为 0 则跳出循环
04
05     LDR r4, [r0], #4    // 取数并自动将数组 A 指针+4
06     LDR r5, [r1], #4    // 取数并自动将数组 B 指针+4
07
08     MUL r6, r4, r5      // 乘法运算
09     ADD r2, r2, r6      // 累加
10
11     SUB r3, r3, #1      // 计数器-1
12     B dot_product_loop  // 跳转回循环

```

RISC-V:

```

01 dot_product_loop:
02     beq s2, zero, end_loop # 计数器为 0 则跳出循环
03
04     lw t1, 0(s0)          # 取数 A[i]
05     lw t2, 0(s1)          # 取数 B[i]
06
07     mul t3, t1, t2        # 乘法运算
08     add s3, s3, t3        # 累加
09
10     addi s0, s0, 4        # 使用独立指令使数组 A 指针+4
11     addi s1, s1, 4        # 使用独立指令使数组 B 指针+4
12     addi s2, s2, -1       # 计数器-1
13     j dot_product_loop   # 跳转回循环

```

4. 实验步骤及预期结果

(1) 实验步骤:

完成 QEMU User 以及对应的交叉编译工具等配置后，我们将分别对两份汇编代码进行编译，同时生成反汇编文件来深入分析内部指令。请依照如下步骤进行实验：

i. ARM 架构代码编译与分析

在终端中依次执行以下命令：

```

01 # 1. 编译 ARM 代码
02 arm-linux-gnueabi-gcc -nostdlib -static 1012-2410668-吕昊远-arm.s -o arm_dot
03
04 # 2. 运行 QEMU
05 /opt/qemu-6.2/bin/qemu-arm ./arm_dot
06
07 # 3. 打印上一条指令的退出码（验证结果）
08 echo $?
09
10 # 4. 生成反汇编文件
11 arm-linux-gnueabi-objdump -d arm_dot > arm_asm.txt

```

ii. RISC-V 架构代码编译与分析

在终端中依次执行以下命令：

```

01 # 1. 编译 RISC-V 代码
02 riscv64-linux-gnu-gcc -nostdlib -static 1012-2410668-吕昊远-riscv.s -o riscv_dot
03
04 # 2. 运行 QEMU
05 /opt/qemu-6.2/bin/qemu-riscv64 ./riscv_dot
06
07 # 3. 打印上一条指令的退出码（验证结果）
08 echo $?
09
10 # 4. 生成反汇编文件
11 riscv64-linux-gnu-objdump -d riscv_dot > riscv_asm.txt

```

在本部分，首先应对代码运行的结果进行验证，确认计算结果与运算逻辑均无误后，再进行以下的分析。

(2) 结果分析：

完成上述操作后，使用文本编辑器分别打开生成的 arm_asm.txt 和 riscv_asm.txt 文件，依照以下步骤进行对比分析：

- 在两个反汇编文件中找到标签 <dot_product_loop>。该区域为点积运算的核心循环部分，是指令集特征的集中体现。
- 统计上述循环体内单次迭代所消耗的指令条数，并进行初步分析。

- 观察两份代码在执行加载数组元素到寄存器并更新数组指针地址这一复合动作时，所调用的指令类型以及相应的寻址逻辑。

(3) 预期结论：

按照上述流程进行分析，将得出以下结论：

i. 代码密度：

在反汇编文件中，ARM 版本的核心循环体包含 8 条指令；而 RISC-V 版本的核心循环体则需要 9 条指令。在本次实验算法的场景下，ARM 指令集表现出更高的代码密度。

ii. 指令差异：

对比反汇编片段进行分析，导致上述指令条数差异的核心原因在于访存指令的设计。ARM 凭借后序自增寻址模式在两次数组访存中节省了 2 条指针更新指令；然而，由于 RISC-V 采用了比较与分支合并的设计，在循环边界检查处比 ARM 少用 1 条指令。

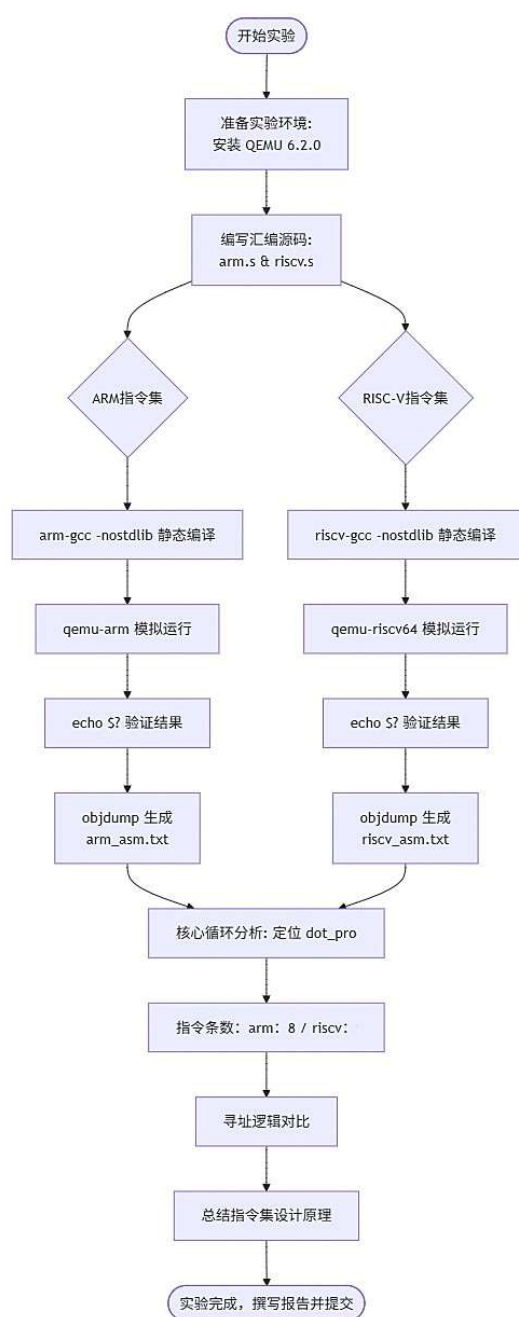
ARM 采用复合寻址模型，即后序自增寻址模式（Post-indexed Addressing）。如指令 `LDR R4, [R0], #4`，该指令在微架构层面仅发生一次译码，却同时完成“将内存基址读入寄存器”与“将基址寄存器数值自增 4”两项操作。而 RISC-V 的访存指令仅支持基础的基址变址寻址（Base+Offset）。可以观察到在代码中，`lw t1, 0(s0)` 仅读取数据，随后使用一条独立的算术逻辑指令 `addi s0, s0, 4` 来进行指针的更新。

iii. 设计原理：

通过以上结果，可以分析得知两种指令集的设计原理：ARM 引入复合寻址模式，一定程度上来自于复杂指令集优化代码体积的理念，通过减

少取指次数和总线带宽占用，来提升特定常规操作的执行效率；而 RISC-V 则是秉持 RISC 的极简主义设计原则，分离了访存与算术逻辑操作（Load/Store 架构），拒绝在访存指令中引入指针更新逻辑，虽然牺牲了部分代码密度，但是大幅降低了处理器流水线和硬件译码器的设计复杂度，有利于优化关键路径的时序，从而实现更高的处理器主频。

5. 实验流程与思路图



6. 实验评估方法

作为指导者，可参考以下几个方面对同学们的实验完成情况进行评估：

i. **实验操作（35%）**：能够正确配置实验环境，成功运行 qemu-arm 和 qemu-riscv64；编写实验所需的代码并编译，输出正确的运算结果，同时依照要求正确完成反汇编工具的使用。可对照附件的反汇编文件（arm_asm.txt、riscv_asm.txt）或对比标准的运行步骤进行评估：

```
1 arm-linux-gnueabi-gcc -nostdlib -static 1012-2410668-吕昊远-arm.s -o arm_dot
2 /opt/qemu-6.2/bin/qemu-arm ./arm_dot
3 echo $?
4 arm-linux-gnueabi-objdump -d arm_dot > arm_asm.txt
5
6 riscv64-linux-gnu-gcc -nostdlib -static 1012-2410668-吕昊远-riscv.s -o riscv_dot
7 /opt/qemu-6.2/bin/qemu-riscv64 ./riscv_dot
8 echo $?
9 riscv64-linux-gnu-objdump -d riscv_dot > riscv_asm.txt
```

ii. **结果分析（40%）**：能够正确统计核心代码的指令条数并分析原因，准确指出反汇编文件具体的核心片段并对其差异进行分析，同时对比寻址模式的指令逻辑，解释其中几条重点的指令对应的操作都是什么。使用适当的语言讲解对比分析的思路与过程。

iii. **知识掌握（20%）**：在报告中准确阐述两种指令集的原理与设计理念差异，自行查找相关资料，对比两种指令集设计的优缺点与适用场景，可以与现实工程进行一定联系。

vi. **进阶要求（5%）**：鼓励同学们在此基础上进行进一步的深入探索，例如：可以自行编写所需的汇编代码，添加额外的功能进行额外的探索学习（如 RISC-V 比较分支合并）；考虑使用其他的分析工具，多方面验证实验

结论；对额外数据进行可视化分析，优化报告表现；阅读相关的参考资料，
结合现实角度进行思考……